

Trading Algorithmique et Arbitrage Statistique (LF et HF)

Partie 2 - Modèles et stratégies tendanciels

Diquel Dos Santos
Head of Fixed Income Quantitative Research at Candriam

Université Lille I - Master 2 Mathématiques et Finance

Mars 2023

Introduction

- Famille de stratégies basée sur la constatation empirique de **persistence des tendances** (trend momentum) à moyen/long terme
- **Tendance** : fonction x_t suffisamment lisse représentant les mouvements long-terme de la série de prix $y_t = P_t$ ou $y_t = \ln(P_t)$

$$y_t = x_t + \epsilon_t = f_t[y_t] + \varepsilon_t$$

$$\sigma(y_t - y_{t-1}) \gg \sigma(x_t - x_{t-1})$$

- **Ce qui nous intéresse** : le signe de la pente de cette tendance

$$\mu_t = \frac{dx_t}{dt} = \frac{1}{\Delta} [x_t - x_{t-1}]$$

Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile
- 5 Filtres L_p
- 6 Filtres de Kalman
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Analyse technique (1)

- Synthèse de décénies d'expérience de marché, de règles empiriques et d'observations de faits stylisés récurrents
- **Fonctions et objectifs :**
 - représenter graphiquement et à l'aide d'indicateurs l'information pertinente de marché
 - reconnaître un certain nombre de formes (patterns) récurrentes et historiquement prédictives
- **Aujourd'hui** : utilisée principalement en complément d'autres analyses, qualitatives et quantitatives
- Beaucoup d'idées et notions importantes introduites par l'analyse technique : reconnaissance de forme, détection de tendances, indicateurs (techniques)...
- L'analyse technique n'est rien d'autre qu'une **méthode graphique de classification supervisée**

Analyse technique (2)

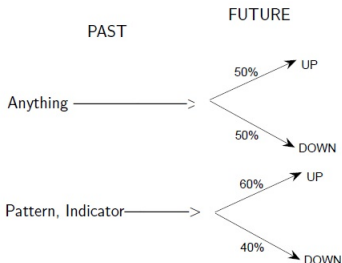
- Plusieurs théories complémentaires : Dow, Elliott...
- **A première vue « ésotériques », mais justifications liées à :**
 - des phénomènes profonds et récurrents liées aux activités de marché
 - la finance comportementale
- **3 principaux objets d'analyse :** les graphes, les patterns et les indicateurs
- L'analyse technique récuse implicitement la forme faible d'efficience de marché

Economic Theory

Prices follow a random walk

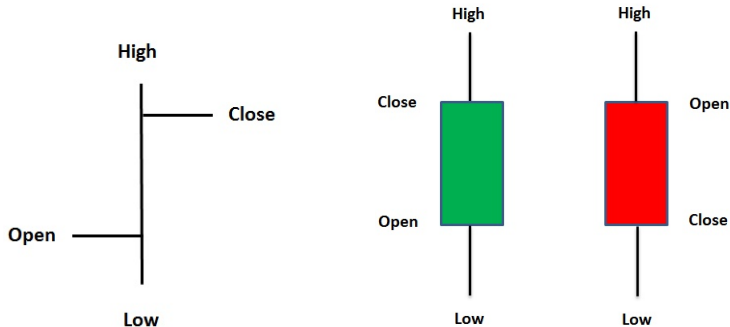
Technical Analysis

Past may influence future



Analyse technique : Graphes

- Outil de base de l'analyse technique
- **Informations additionnelles** : Open-High-Low-Close (OHLC)
- **2 principaux graphes** :
 - en barres
 - en chandeliers (japonais)



Graphes : Barres OHLC



Graphes : Barres OHLC



Analyse technique : Patterns (1)

- 2 notions importantes liées au **rapport relatif entre offre et demande** :
 - **Ligne de support** : ligne reliant les prix bas limites (seuil d'activité minimal de la demande)
 - **Ligne de résistance** : ligne reliant les prix hauts limites (seuil d'activité minimal de la demande)



Analyse technique : Patterns (2)

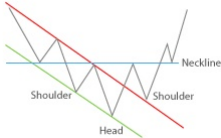
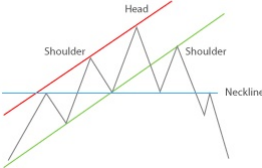
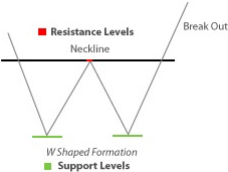
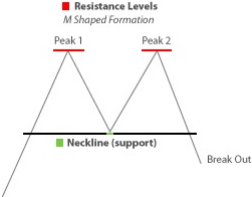
- Ces lignes permettent :
 - **d'identifier des canaux tendanciels**
 - **de délimiter des patterns** bien répertoriés qui sont suivis historiquement de tendances bien définies
- **Objectif final** : générer des signaux de trading (Bullish/Bearish market)
- **3 principales classes de patterns** :
 - **Continuation Patterns** : patterns de confirmation de tendance (pas de changement structurel entre offre et demande)
 - **Reversal Patterns** : patterns de rupture de tendance (inversion majeure du rapport de force entre offre et demande)
 - **Vagues d'Elliott**

Continuation Patterns

Outre les canaux tendanciels haussiers et baissiers, les plus importants sont :

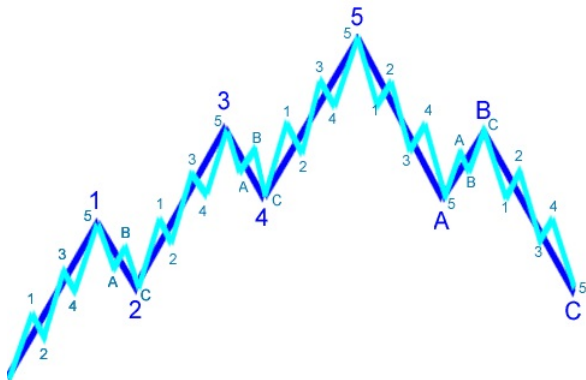
Pattern	Bullish Signal	Bearish Signal
Triangle (Penant)	Ascending Triangle	Descending Triangle
Rectangle (Flag)	Bullish Rectangle	Bearish Rectangle

Reversal Patterns

Pattern	Bullish Signal	Bearish Signal
Head and Shoulders	HEAD AND SHOULDERS BOTTOM 	HEAD AND SHOULDERS TOP 
	DOUBLE BOTTOM FORMATION 	DOUBLE TOP FORMATION 
Double top et Double Bottom		

Vagues d'Elliott (1)

- Principes empiriques => **2 phases successives** :
 - **Impulse Phase** : les prix bougent suivant 5 vagues (1-2-3-4-5) dans la tendance dominante
 - **Corrective Phase** : puis 3 vagues (A-B-C) dans le sens contraire



Vagues d'Elliott (2)

- **Caractère fractal** de ce pattern
- Elliott isole historiquement **9 cycles d'horizons très différents** :
 - 1 Grand Supercycle (en centaines d'années)
 - 2 Supercycle
 - 3 Cycle
 - 4 Primary
 - 5 Intermediate
 - 6 Minor
 - 7 Minute
 - 8 Minuette
 - 9 Subminuette (quelques minutes)

Analyse technique : Indicateurs

- Compléments à l'analyse graphique
- **4 grandes familles :**
 - **Indicateurs de suivi de tendance** : différence de moyennes mobiles ST-LT simples ou exponentielles (MACD) + bandes de Bollingers (1 ou 2 écart-types)
 - **Oscillateurs** : détectent des situations de tension menant à un renversement de tendance (ex : RSI, rapport normalisé nb hausses/nb baisses sur une période)
 - **Indicateurs de sentiment de marché** : sondages, call/put ratio...
 - **Indicateurs de flux et de volume**

Indicateurs : Différence de moyennes mobiles (1)

- Indicateurs les plus utilisés
- **Bon compromis entre simplicité de calcul et efficacité prédictive**
- Ecart relatif entre 2 moyennes mobiles homogènes :

$$I_t = MA_t^{ST} - MA_t^{LT}$$

- La moyenne mobile ST est considérée plus réactive que la LT
- **Le signe de l'indicateur donne la tendance de la série :**
 - haussière si I_t est positif
 - baissière si I_t est négatif
- **Changement de signe de l'indicateur = signal de rupture de tendance**
- **Graphiquement :**
 - **Golden Cross** : la moyenne mobile court terme MA_t^{ST} croise la moyenne mobile long terme MA_t^{LT} par le bas => la tendance baissière devient haussière => **Signal Long**
 - **Death Cross** : la moyenne mobile court terme MA_t^{ST} croise la moyenne mobile long terme MA_t^{LT} par le haut => la tendance haussière devient baissière => **Signal Short**

Indicateurs : Différence de moyennes mobiles (2)



Indicateurs : Différence de moyennes mobiles (3)

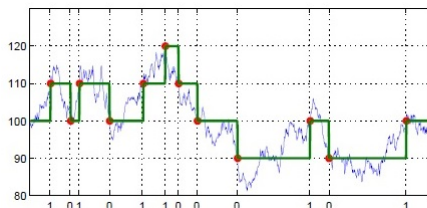
- **Moyennes mobiles homogènes les plus utilisées** : 20-60 jours et 50-200 jours
- **Moyennes mobiles exponentielles** : permettent de donner plus de poids aux observations récentes
- **MACD** :

$$MACD_t = EMA_t^{ST} (12) - EMA_t^{LT} (26)$$

- moyenne mobile exponentielle court terme
- les signaux de rupture de tendance sont donnés par le croisement de $MACD_t$ avec EMA_t (9).

Reconnaissance de formes : Binary Strings (1)

- **Tentative de formalisation quantitative de l'analyse technique**
- **Approche :**
 - digitaliser la série y_t à l'aide de 0 ou 1 en fonction des hausses et des baisses observées
 - identifier des séquences ayant une forte probabilité d'être suivi d'une hausse ou d'une baisse => signal d'achat ou de vente
- **Digitalisation** : filtrage de la série pour ne garder que l'information pertinente des mouvements de prix tout en préservant la volatilité de la série initiale



— up → 1

— down → 0

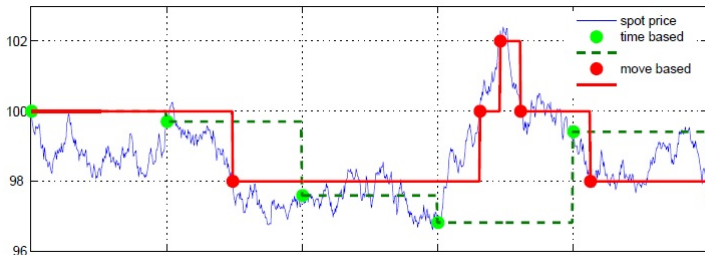
Sequence:

1 0 1 0 1 1 0 0 0 1 0 1

Reconnaissance de formes : Binary Strings (2)

- 2 schémas de digitalisation :

- Time Based** : on discrétise la série à périodicité constante $\Delta t = cte$
 - Move Based** : on discrétise la série sur un autre niveau dès que l'actif y_t a bougé d'un montant supérieur à $\Delta y = \alpha = cte$ (changements de niveau d'activité sur le marché)
- Même problématique que pour le rebalancement de stratégies de couverture
 - On peut montrer que **le schéma Move Based est le plus efficace** en terme de convergence de la série discrétisée vers la série initiale lorsque $\alpha \rightarrow 0$



Reconnaissance de formes : Binary Strings (3)

- **Etape 1** : on découpe la série digitalisée en séquences de digits (Binary Strings) X_i de taille n auxquelles on associe le digit ϵ_i qui lui fait suite

$$\begin{array}{c} X_i \quad \epsilon_i \\ \underbrace{011001} \quad \underbrace{1} \end{array}$$

- **Etape 2** : pour un BS donné $Y = (y_1 \dots y_k \dots y_n)$ on calcule la probabilité historique $p = P(\epsilon = 1/BS = Y)$
- **Problème** : nécessite des historiques très longs
- **Solution** : on considère plutôt une probabilité qui pondère les ϵ_i inversement à la distance $d_i = d(X_i, Y)$

$$p = P(\epsilon = 1/BS = Y) = \frac{\sum e^{-\lambda d_i} \epsilon_i}{\sum e^{-\lambda d_i}}$$

$$d_i = d(X_i, Y) = \sum_{k=1}^n (x_k^i - y_k)^2 \alpha^{n-k}$$

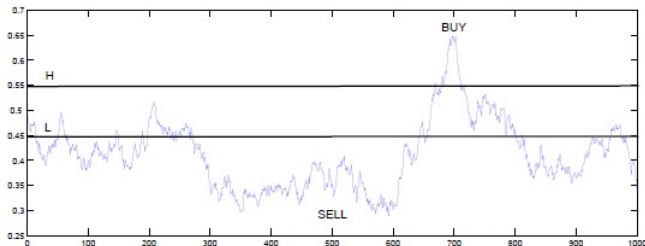
Reconnaissance de formes : Binary Strings (4)

- **Signaux de trading :**

- $p > H \Rightarrow$ signal Long
- $p < L \Rightarrow$ signal short

- If $p < L$:
sell

- If $p > H$:
buy



Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile
- 5 Filtres L_p
- 6 Filtres de Kalman
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Régression locale (1)

- A la date τ , on considère une **famille de $m + 1$ fonctionnelles temporelles** :

$$F_{\tau}(\tau - t) = (f_0^{\tau}(\tau - t) \quad \dots \quad f_j^{\tau}(\tau - t) \quad \dots \quad f_m^{\tau}(\tau - t))$$

- **Régression locale autour de τ** :

$$y_t = x_t + \epsilon_t$$

$$y_t = \sum_{j=0}^m \beta_j^{\tau} f_j^{\tau}(\tau - t) + \varepsilon_t$$

$$y_t = F_{\tau}(\tau - t) \beta_{\tau} + \varepsilon_t$$

- **Estimation par critère des moindres carrés pondérés** sur un historique de longueur $n + 1 \geq m + 1$ ($t_n = n = \tau$) :

$$\hat{\beta}_{\tau} = \underset{\beta_{\tau}}{\operatorname{argmin}} \sum_{i=0}^n w_i^{\tau} (y_i - F_{\tau}(\tau - i) \beta_{\tau})^2$$

$$\hat{\beta}_{\tau} = \underset{\beta_{\tau}}{\operatorname{argmin}} (Y_{\tau} - F_{\tau} \beta_{\tau})^T W_D^{\tau} (Y_{\tau} - F_{\tau} \beta_{\tau})$$

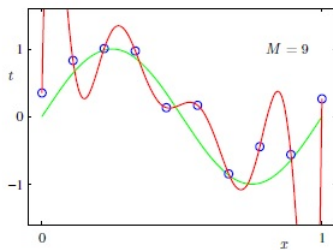
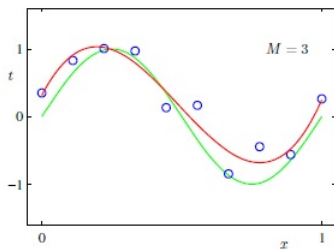
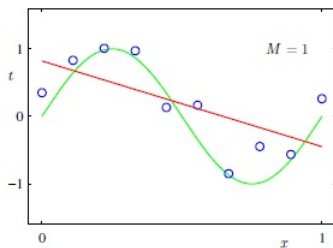
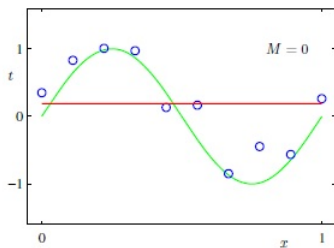
Régression locale (2)

- **Solution explicite :**

$$\hat{\beta}_{\tau} = [F_{\tau}^T W_D F_{\tau}]^{-1} F_{\tau}^T W_D^T Y_{\tau}$$

- En pratique, on choisit souvent une **base polynomiale** pour $F_{\tau}(\tau - t)$:
 - **Base polynomiale canonique** : $f_j(t) = t^j$
 - **Base polynomiale de Legendre** : $f_j(t) = \frac{t(t-1)(t-2)\dots(t-j)}{j!}$
- **Compromis biais-variance uniquement piloté par m :**
 - $m + 1 = n + 1$: interpolation parfaite $\Rightarrow$ over-fitting + oscillations aux bords et en particulier en τ
 - $m = 0$ ou $m = 1$: variance faible mais biais très élevé
- **Un bon compromis** : $m = 2$ ou $m = 3$

Régression locale (3)



Régression locale (4)

- Le cas $m = 3$ conduit aux splines cubiques $s(t)$
- **Splines cubiques** : suite de $p - 1$ polynômes cubiques C^2 par morceaux passant exactement par $2 \leq p \leq n + 1$ points

$$x_t = s_\tau(\tau - t) = \begin{cases} s_1^\tau(\tau - t) & t \in [t_1, t_2[\\ \vdots & \vdots \\ s_k^\tau(\tau - t) & t \in [t_k, t_{k+1}[\\ \vdots & \vdots \\ s_{p-1}^\tau(\tau - t) & t \in [t_{p-1}, t_p[\end{cases}$$

- **Propriété** : toute spline cubique peut s'écrire comme une combinaison linéaire de $p + 3$ B-Splines $B_\tau(\tau - t)$
- **On se ramène ainsi au cadre précédent** :

$$\begin{aligned} s_\tau(\tau - t) &= \sum_{j=0}^{p+2} \beta_j^\tau B_j^\tau(\tau - t) \\ &= B_\tau(\tau - t) \beta_\tau \end{aligned}$$

Méthodes de lissage : EWMA

- Méthodes permettant d'améliorer le filtrage de la série et de régulariser la fonctionnelle temporelle
- Introduction d'hyperparamètres pilotant le compromis biais-variance
- **2 méthodes complémentaires :**
 - choix judicieux des pondérations w_i^τ pour l'estimation MCP
 - introduction d'une fonction de pénalité
- **Lissage exponentiel (EWMA)** : on privilégie les observations récentes => poids décroissant exponentiellement avec le temps

$$w_i^\tau = \lambda^{\tau-i}$$

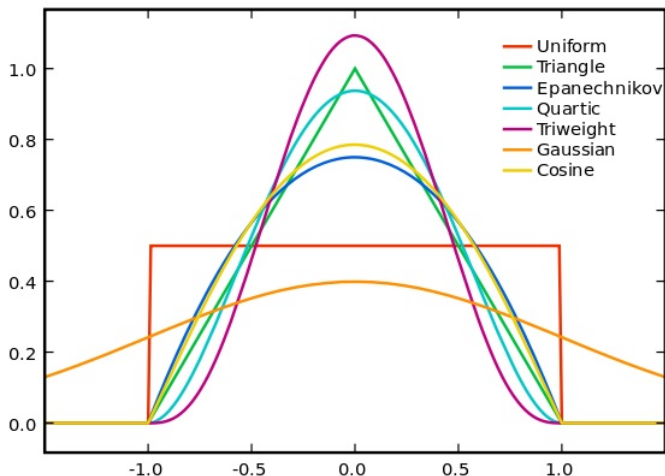
Méthodes de lissage : Kernel smoothing (1)

- **Kernel** : densité de probabilité symétrique vérifiant les deux propriétés suivantes :
 - $\int_{-\infty}^{+\infty} K(t) dt = 1$
 - $K(-t) = K(t)$
- **Autres propriétés** :
 - si K est un kernel alors $K_h(t) = \frac{1}{h} K\left(\frac{t}{h}\right)$ également ($h > 0$)
 - si le support de K est $[-1, 1]$ alors celui de K_h est $[-h, h]$
- **Kernel smoother** : kernel K à support dans $[-1, 1]$ + hyperparamètre $h > 0$

$$w_i^\tau = \frac{1}{h} K_\tau \left(\frac{\tau - i}{h} \right)$$

- Ce type de pondération pénalise aussi les données les plus anciennes

Méthodes de lissage : Kernel smoothing (2)



Méthodes de lissage : Fonction de pénalité (1)

- Permet également de contrôler le compromis biais-variance/over-fitting et de régulariser la fonctionnelle temporelle
- **Pénalisation des paramètres non-significatifs :**

$$\hat{\beta}_{\tau}(\lambda) = \underset{\beta_{\tau}}{\operatorname{argmin}} \sum_{i=0}^n w_i^T (y_{\tau-i} - F_{\tau}(\tau-i)\beta_{\tau})^2 + \lambda \|\beta_{\tau}\|_2^2$$

$$\hat{\beta}_{\tau}(\lambda) = \underset{\beta_{\tau}}{\operatorname{argmin}} (Y_{\tau} - F_{\tau}\beta_{\tau})^T W_D^T (Y_{\tau} - F_{\tau}\beta_{\tau}) + \lambda \|\beta_{\tau}\|_2^2$$

- **Solution explicite :**

$$\hat{\beta}_{\tau}(\lambda) = [F_{\tau}^T W_D F_{\tau} + \lambda I]^{-1} F_{\tau}^T W_D^T Y_{\tau}$$

Méthodes de lissage : Fonction de pénalité (2)

- **Pénalisation des irrégularités de la fonction :**

$$\hat{\beta}_{\tau}(\lambda) = \underset{\beta_{\tau}}{\operatorname{argmin}} \sum_{i=0}^n w_i^{\tau} (y_i - F_{\tau}(\tau - i) \beta_{\tau})^2 + \lambda \int_0^{\tau} w_t^{\tau} (F_{\tau}''(\tau - t) \beta_{\tau})^2 dt$$

$$\hat{\beta}_{\tau}(\lambda) = \underset{\beta_{\tau}}{\operatorname{argmin}} (Y_{\tau} - F_{\tau} \beta_{\tau})^T W_D^{\tau} (Y_{\tau} - F_{\tau} \beta_{\tau}) + \lambda \int_0^{\tau} w_t^{\tau} (F_{\tau}''(\tau - t) \beta_{\tau})^2 dt$$

- On introduit la matrice $V = (v_{ij})$ de taille $m \times m$ avec
 $v_{ij} = \int_0^{\tau} w_t^{\tau} (f_i^{\tau}(\tau - t))'' (f_j^{\tau}(\tau - t))'' dt$
- Le problème peut se réécrire ainsi :

$$\hat{\beta}_{\tau}(\lambda) = \underset{\beta_{\tau}}{\operatorname{argmin}} (Y_{\tau} - F_{\tau} \beta_{\tau})^T W_D^{\tau} (Y_{\tau} - F_{\tau} \beta_{\tau}) + \lambda \beta_{\tau}^T V \beta_{\tau}$$

- **Solution explicite :**

$$\hat{\beta}_{\tau}(\lambda) = [F_{\tau}^T W_D^{\tau} F_{\tau} + \lambda V]^{-1} F_{\tau}^T W_D^{\tau} Y_{\tau}$$

Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage**
- 4 Filtres à moyenne mobile
- 5 Filtres L_p
- 6 Filtres de Kalman
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Bases fréquentielles : Fourier (1)

- **Représentation dans la base de Fourier :**

$$y_t = \sum_{k=0}^n \hat{y}(k) e^{i2\pi t \frac{k}{n}}$$

$$y_t = \sum_{k=0}^n w_k e^{i2\pi t \frac{k}{n}}$$

$$y_t = F^{-1}(\hat{y}(k)) = F^{-1}(w_k)$$

- Les coefficients $w_k = \hat{y}(k)$ de cette base résultent de la transformation de Fourier F opérée sur la série y_t :

$$w_k = \hat{y}(k) = F(y_t) = \frac{1}{n} \sum_{t=0}^n y_t e^{-i2\pi k \frac{t}{n}}$$

- **Problème** : dimension de la base = taille de la fenêtre d'historique => over-fitting
- **Solution** : débruitage de la série

Bases fréquentielles : Fourier (2)

- **Etapes de débruitage :**

- ① **Seuillage (shrinkage)** : on applique la fonction de seuillage $d(w)$ aux coefficients $w_k = \hat{y}(k)$ pour éliminer les coefficients les plus faibles
- ② **Recomposition** : application de la transformée de Fourier inverse F^{-1} sur les coefficients seuillés $d(w_k)$:

$$y_t = F^{-1}(d(w_k)) + \varepsilon_t$$

$$y_t = \sum_{k=0}^n d(w_k) e^{i2\pi t \frac{k}{n}} + \varepsilon_t$$

- **Problèmes :**

- la théorie spectrale n'est pas valide pour les processus non-stationnaires comme les séries de prix
- cette approche localise très mal en temps les signaux LF => détecte très mal les changements de tendance

- **Solution** : bases d'ondelettes

Méthodes de seuillage

- **Seuillage dur (hard shrinkage) :**

$$d(w) = w \cdot 1_{|w| > w^+} = \begin{cases} w & |w| > w^+ \\ 0 & |w| \leq w^+ \end{cases}$$

- **Seuillage doux (soft shrinkage) :**

$$d(w) = \text{sign}(w) \cdot (|w| - w^+)^+ = \begin{cases} w - w^+ & w \geq w^+ \\ w + w^+ & w \leq -w^+ \\ 0 & |w| \leq w^+ \end{cases}$$

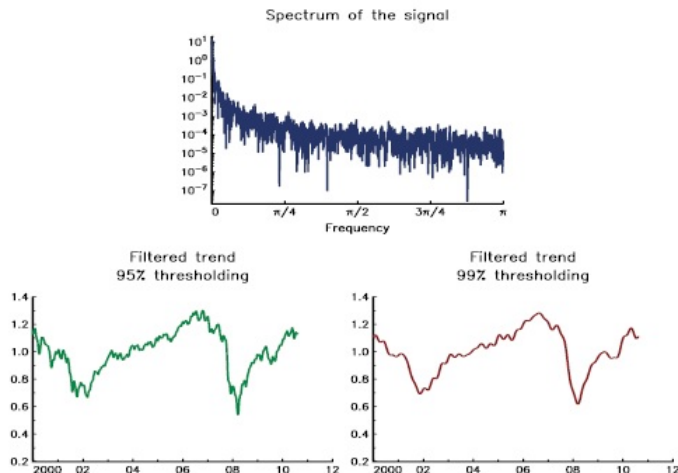
- **Seuillage semi-doux (semi-soft shrinkage) :**

$$d(w) = \begin{cases} 0 & |w| \leq w^- \\ \text{sign}(w) \frac{w^+ (|w| - w^-)}{w^+ - w^-} & w^- < |w| \leq w^+ \\ w & |w| > w^+ \end{cases}$$

- **Seuillage par quantile (quantile shrinkage) :** seuillage dur où w^+ est le q^{th} quantile des coefficients $|w_k|$

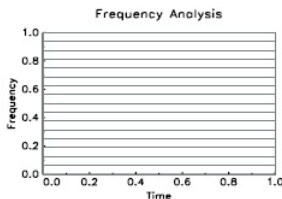
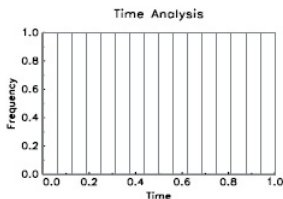
Bases fréquentielles : Fourier (3)

Filtrage spectral à l'aide d'un seuillage par quantile respectivement à 95% et 99%



Bases temporelles vs Bases fréquentielles

- **Compromis à trouver** entre :
 - **Base temporelle** : détecte correctement les anomalies de marché (crashes, sauts...) mais ne modélise pas les répétitions observées sur la série
 - **Base fréquentielle** : modélise correctement les phénomènes récurrents mais détectent très mal les bosses ou sauts de marché
- **Supports différents** dans l'espace temps-fréquence :
 - **Base temporelle** : sur chaque pavé (temporel) la base considérée est valable pour toutes les fréquences du spectre
 - **Base fréquentielle** : sur chaque pavé (spectral) la base considérée est valable à toute date



Bases mixtes

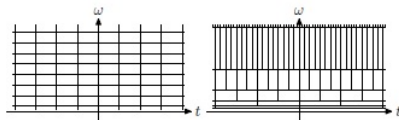
- **2 bases mixtes** proposées et construites à partir d'une fonction racine g_0 :
 - **Approche temps-fréquence (Fourrier à fenêtre)** : les supports de fonction sont des rectangles tous identiques et la largeur temporelle Δt est indépendante de la fréquence

$$g_{u,w}(t) = e^{i\omega t} g_0(t - u)$$

- **Approche temps-échelle (ondelettes)** : la largeur temporelle $\Delta t = s$ des fonctions est inversement proportionnelle à la fréquence $w = \frac{cte}{s} \Rightarrow$ plus la fréquence est élevée plus la résolution des pavages est importante

$$g_{u,s}(t) = \frac{1}{\sqrt{s}} g_0\left(\frac{t - u}{s}\right)$$

- L'approche par ondelettes est supérieure (propriétés + temps de calculs)



(a) Temps-fréquence

(b) Temps-échelle

Ondelettes « Père » et « Mère » (1)

- **Construction des bases d'ondelettes au sein $L^2(\mathbb{R})$** à partir d'une ondelette dite « père » $\phi(t)$ et une ondelette associée dite « mère » $\psi(t)$
- **Ondelette dite « père » :**

$$\begin{cases} \int_{-\infty}^{+\infty} \phi(t) &= 1 \\ \|\phi(t)\| &= 1 \end{cases}$$

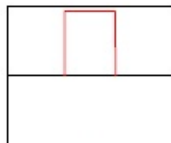
- **Ondelette dite « mère » :** construite à partir de l'ondelette dite « père »

$$\psi(t) = \phi(2t) - \phi(2t - 1)$$

$$\begin{cases} \int_{-\infty}^{+\infty} \psi(t) &= 0 \\ \|\psi(t)\| &= 1 \end{cases}$$

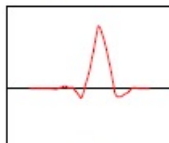
Ondelettes « Père » et « Mère » (2)

Haar Basis

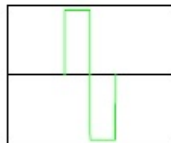


$\phi(x)$

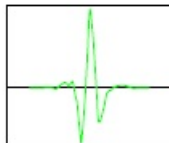
Symmlet Basis



$\phi(x)$



$\psi(x)$



$\psi(x)$

Ondelettes filles (1)

- Par **dilatation et translation**, on construit 2 familles d'ondelettes dites « filles » à partir des ondelettes « père » et « mère » et ayant les mêmes propriétés :

$$\phi_{s,u}(t) = \frac{1}{\sqrt{s}} \phi\left(\frac{t-u}{s}\right)$$

$$\psi_{s,u}(t) = \frac{1}{\sqrt{s}} \psi\left(\frac{t-u}{s}\right)$$

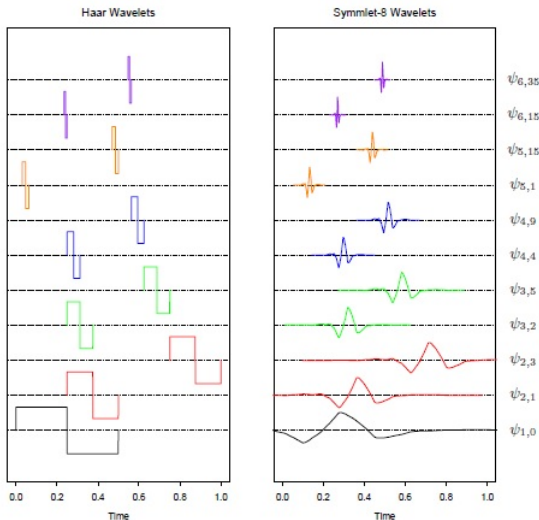
- Echantillonnage dyadique** $s = 2^{-j}$ et $u = ks \Rightarrow$ **ondelettes discrètes** :

$$\phi_{j,k}(t) = 2^{\frac{j}{2}} \phi(2^j t - k)$$

$$\psi_{j,k}(t) = 2^{\frac{j}{2}} \psi(2^j t - k)$$

Ondelettes filles (2)

Ondelettes « filles » à différentes translations et dilatations



Ondelettes : Analyse multirésolution (1)

- Pour une valeur de dilatation j , le support des ondelettes $(\psi_{j,k}(t))_{k \in \mathbb{Z}}$ est 2^j fois plus petit que pour l'ondelette « mère » $\psi(t) \Rightarrow$ **analyse multirésolution**

Théorème

Pour tout $j \in \mathbb{Z}$, les familles $(\phi_{j,k}(t))_{k \in \mathbb{Z}}$ et $(\psi_{j,k}(t))_{k \in \mathbb{Z}}$ définissent une base orthonormale d'un certain espace vectoriel $L^2(\mathbb{R})$ respectivement V_j et W_j

• Principales propriétés :

- $\bigcup_{j \in \mathbb{Z}} V_j$ et $\bigcup_{j \in \mathbb{Z}} W_j$ sont tous deux denses dans $L^2(\mathbb{R})$: $\bigoplus_{j \in \mathbb{Z}} V_j = L^2(\mathbb{R})$ et $\bigoplus_{j \in \mathbb{Z}} W_j = L^2(\mathbb{R})$
- $\bigcap_{j \in \mathbb{Z}} V_j = \{0\}$ et $\bigcap_{j \in \mathbb{Z}} W_j = \{0\}$
- Les espaces $(V_j)_{j \in \mathbb{Z}}$ sont emboîtés : $V_j \subset V_{j+1}$
- W_j est le complémentaire de V_j dans V_{j+1} autrement dit $V_{j+1} = V_j \oplus W_j$
- $V_j = V_0 \oplus \bigoplus_{k \in [0, j-1]} W_k$

Ondelettes : Analyse multirésolution (2)

- On dit que l'espace V_j a une résolution $\frac{1}{2^j}$ car il décrit :
 - $L^2(\mathbb{R})$ 2 fois plus finement que V_{j-1}
 - 2^j plus finement que V_0
- L'espace W_j correspond aux détails V_j dans V_{j+1}
- Série y_t :
 - on considère n observations historiques en τ
 - on définit J tel que $n = 2^J$
 - on approxime $L^2(\mathbb{R})$ par V_J qui définit donc une résolution fonctionnelle $\frac{1}{2^J}$
- Décomposition de l'espace V_j :

$$V_J = V_0 \oplus \bigoplus_{j \in [0, J-1]} W_j$$

- $L^2(\mathbb{R})$ est donc vue comme l'espace V_0 auquel on ajoute une série de détails de résolution de plus en plus fines jusqu'à la résolution $\frac{1}{2^{J-1}}$

Décomposition en base d'ondelettes (1)

- On tronque les bases de V_0 et des $(W_j)_{j \in [0, J-1]}$:

$$B_{V_0} = \{\phi_{0,0}(t) = \phi(t)\}$$

$$B_{W_j} = \{\psi_{j,k}(t) \mid k \in [0, 2^j - 1]\}$$

- Décomposition de y_t dans la base d'ondelettes $B_{V_0} \cup \bigcup_{j \in [0, J-1]} B_{W_j}$:

$$y_t = w_0 \cdot \phi(t) + \sum_{j=0}^{J-1} \sum_{k=0}^{2^j-1} w_{j,k} \psi_{j,k}(t)$$

- Propriétés des coefficients :

- Nombre** : $2^J - 1$ coefficients $w_{j,k}$ soit $2^J = n$ en tout en comptant w_0
- Calcul** : les coefficients w_0 et $w_{j,k}$ ne sont rien d'autres que la **projection orthogonale** de y_t sur respectivement $\phi(t)$ et $\psi_{j,k}(t)$

Décomposition en base d'ondelettes (2)

- Les coefficients définissent ainsi une **transformation d'ondelettes** W sur la **série** y_t à l'instar de la transformée de Fourier :

$$w_0 = \langle y_t, \phi(t) \rangle = \sum_{t=0}^n y_t \phi(t)$$

$$w_{j,k} = W(y_t) = \langle y_t, \psi_{j,k}(t) \rangle = \sum_{t=0}^n y_t \psi_{j,k}(t)$$

- Filtrage du signal** : application d'une **méthode de seuillage** $d(w)$ + **recomposition** :

$$y_t = d(w_0) \cdot \phi(t) + \sum_{j=0}^{J-1} \sum_{k=0}^{2^j-1} d(w_{j,k}) \psi_{j,k}(t) + \varepsilon_t$$

- Calcul des coefficients** :
 - Projection vectorielle** = > trop long
 - Algorithme pyramidal tirant partie des caractéristiques d'imbrication des ondelettes** => calculs plus rapides encore que FFT

Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile**
- 5 Filtres L_p
- 6 Filtres de Kalman
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Filtres à moyenne mobile : Définition

- **Filtre** : transformation mathématique $\mathfrak{F}$ qui associe au processus entrant y_t un processus sortant x_t

$$y_t \xrightarrow[\text{input}]{} \mathfrak{F}_t \xrightarrow[\text{output}]{} x_t$$

- **Définition très générale** : les projections temporelles ou fréquentielles peuvent être vues comme des filtres hautement non-linéaires
- **Filtres à moyenne mobile** : sous-classe de filtres

$$x_t = \sum_{i=0}^{n-1} \mathcal{L}_i y_{t-i}$$

- **Propriétés** :

- linéaires
- invariants dans le temps
- causaux : filtres asymétriques ne considérant que les valeurs présentes et passées de y_t
- finis sur un support historique glissant de longueur $T = n\Delta$
- ses coefficients $\mathcal{L}_i$ constituent un kernel de pondérations : $\sum_{i=0}^{+\infty} \mathcal{L}_i = 1$

Estimation de μ_t

- **Analyse harmonique du spectre et de la fonction de gain** : les filtres à moyenne mobile ne sont rien d'autres que des filtres **pas-bas** => **conservation des basses fréquences et donc des cycles longs**
- **Ce qui nous intéresse** : la pente de la tendance et son estimateur

$$\hat{\mu}_t = \frac{dx_t}{dt} = \frac{1}{\Delta} [x_t - x_{t-1}]$$

- **Estimateur possible** : moyenne mobile des rendements réalisés

$$R_t = y_t - y_{t-1}$$

$$\hat{\mu}_t = \frac{1}{\Delta} \sum_{i=0}^{n-1} \mathcal{L}_i R_{t-i}$$

- En introduisant $\ell_i = \mathcal{L}'_i$:

$$\hat{\mu}_t = \frac{1}{\Delta} \sum_{i=0}^{n-1} \ell_i y_{t-i}$$

Dérivée et Primitive de filtres

- Dérivée ℓ_i en fonction de $\mathcal{L}_i$:

$$\ell_i = \begin{cases} \mathcal{L}_0 & \text{si } i = 0 \\ \mathcal{L}_i - \mathcal{L}_{i-1} & \text{si } i \in [1, n-1] \\ -\mathcal{L}_{n-1} & \text{si } i = n \end{cases}$$

- Primitive $\mathcal{L}_i$ en fonction de ℓ_i :

$$\mathcal{L}_i = \begin{cases} \ell_0 & \text{si } i = 0 \\ \ell_i + \mathcal{L}_{i-1} & \text{si } i \in [1, n-1] \\ -\ell_{n+1} & \text{si } i = n \end{cases}$$

Estimateur biaisé

- Très employés en pratique mais attention : **c'est un estimateur biaisé !**
- Supposons par exemple que les prix suivent un modèle log-normal :

$$\frac{dP_t}{P_t} = \mu_t dt + \sigma_t dW_t$$

- Lemme d'Itô $\Rightarrow$ dynamique des rendements R_t :

$$R_t = dy_t = \left(\mu_t - \frac{1}{2} \sigma_t^2 \right) dt + \sigma_t dW_t$$

- En supposant μ_t et σ_t constant sur la fenêtre d'estimation $[t - n, t]$
l'estimateur non-biaisé de μ_t est en réalité :

$$\hat{\mu}_t = \frac{1}{2} \sigma_t^2 + \frac{1}{\Delta} \sum_{i=0}^{n-1} \mathcal{L}_i R_{t-i}$$

Facteurs d'erreur

- Les deux sources d'erreur de l'estimateur $\hat{\mu}_t$: biais et variance

$$\begin{aligned}MSE(\hat{\mu}_t) &= E[(\hat{\mu}_t - \mu_t)^2] \\&= E[(\hat{\mu}_t - E[\hat{\mu}_t])^2] + (E[E[\hat{\mu}_t]] - \mu_t)^2 \\&= Var(\hat{\mu}_t) + (Bias(\hat{\mu}_t))^2\end{aligned}$$

- Les 2 principaux facteurs d'erreur :

- la taille de la fenêtre d'estimation n
- la volatilité du signal σ_t

Erreur	Biais	Variance
Taille de la fenêtre n	croissant	décroissant
Volatilité σ_t	croissant	croissant

Différences de moyennes mobiles ST-LT (1)

- Estimateurs de μ_t très utilisés en pratique
- **Justification théorique** :
 - Considérons le filtre à moyenne mobile homogène sur une fenêtre de taille n :

$$x_t^n = \hat{y}_t^n = \frac{1}{n} \sum_{i=0}^{n-1} y_{t-i}$$

- Dans un cadre log-normal, on peut montrer que la moyenne mobile $ST(n_2)$ - $LT(n_1)$ s'écrit :

$$E[\hat{y}_t^{n_2} - \hat{y}_t^{n_1}] = E[x_t^{n_2} - x_t^{n_1}] = \frac{n_1 - n_2}{2} \left(\mu_t - \frac{1}{2} \sigma_t^2 \right) \Delta$$

- Ceci justifie l'utilisation de l'estimateur (biaisé) :

$$\hat{\mu}_t = \frac{1}{\Delta} \frac{2}{n_1 - n_2} (\hat{y}_t^{n_2} - \hat{y}_t^{n_1})$$

- Rq : cet estimateur est de la forme :

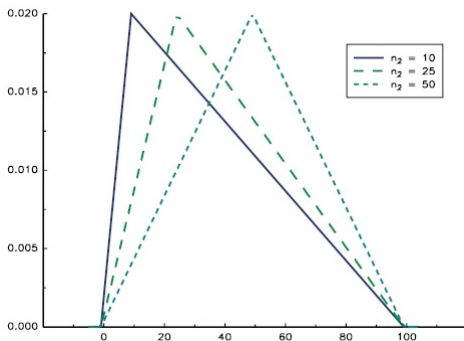
$$\hat{\mu}_t = \frac{1}{\Delta} \sum_{i=0}^{n-1} \ell_i y_{t-i}$$

Différences de moyennes mobiles ST-LT (2)

- En appliquant l'opérateur de primitive, on peut réécrire cet estimateur sous la forme :

$$\hat{\mu}_t = \frac{1}{\Delta} \sum_{i=0}^{n-1} \mathcal{L}_i R_{t-i}$$

- Pondérations $\mathcal{L}_i$ pour une différence de moyennes mobiles ($n_1 = 100$) :**



Filtres à moyenne mobile simple (1)

- Certains **filtres à moyenne mobile simple** ont la même forme que ces différencesis que d'over-fitting associés, la présence de et les de moyennes mobiles
- **Filtre triangulaire** : a la même forme que la différence de moyenne mobiles
 $n_1 = n$ et $n_2 = \frac{n}{2}$

$$\ell_i = \frac{4}{n^2} \text{sign} \left(\frac{n}{2} - i \right)$$

$$\mathcal{L}_i = \frac{4}{n^2} \left(\frac{n}{2} - \left| i - \frac{n}{2} \right| \right)$$

- **Filtre asymétrique** :

$$\mathcal{L}_i = \frac{2}{n^2} (n - i)$$

$$\hat{\mu}_t = \frac{2}{n} \left(y_t - \frac{1}{n} \sum_{i=0}^{n-1} y_{t-i} \right)$$

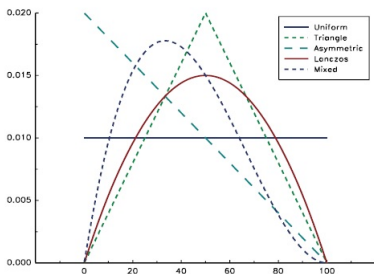
Filtres à moyenne mobile simple (2)

- **Filtre de Lanczos** : se base sur la dérivée du même nom
 $\left(\frac{d^L f(x)}{dx} = \lim_{\varepsilon \rightarrow 0} \frac{3}{2\varepsilon^3} \int_{-\varepsilon}^{\varepsilon} t f(x+t) dt\right)$ qui permet de calculer une
« pseudo-dérivée » à des points où la fonction n'est pas dérivable

$$\ell_i = \frac{12}{n^3} \left(\frac{n}{2} - i \right)$$

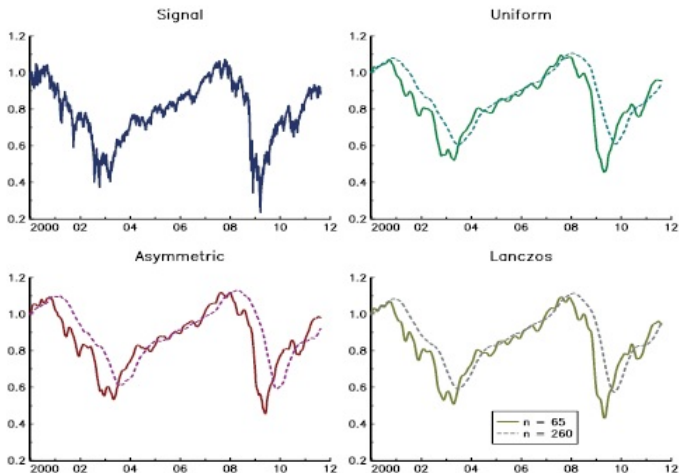
$$\mathcal{L}_i = \frac{6}{n^3} i(n-i)$$

- **Filtre mixte** : convolution des filtres asymétrique et de Lanczos



Filtres à moyenne mobile simple (3)

Filtrage d'un historique de l'indice S&P à l'aide de filtres à moyenne mobile simple



Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile
- 5 Filtres L_p**
- 6 Filtres de Kalman
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Filtres L_p : Définition

- Estimation de la tendance x_t par **MCO pénalisé par la norme L_p de la différentielle seconde de x_t** :

$$x = \underset{x}{\operatorname{argmin}} \frac{1}{2} \|y - x\|_2^2 + \lambda \|Dx\|_p^p$$

- $y = (y_1 \dots y_t \dots y_n)^T$ et $x = (x_1 \dots x_t \dots x_n)^T$
- L'opérateur « différentielle seconde » D** est la matrice $(n-2) \times n$ suivante :

$$D = \begin{pmatrix} 1 & -2 & 1 & & & \\ & 1 & -2 & 1 & & \\ & & & \ddots & & \\ & & & & 1 & -2 & 1 \\ & & & & & 1 & 2 & 1 \end{pmatrix}$$

- L'hyperparamètre $\lambda > 0$ pilote le compromis biais-variance (over-fitting)
- Principaux filtres L_p utilisés en pratique** : filtre L_2 et filtre L_1

Filtres L_2 (1)

- Appelé aussi **filtre de Hodrick-Prescott** (HP Filter), c'est le filtre L_p le plus utilisé en pratique :

$$\begin{aligned}x &= \underset{x}{\operatorname{argmin}} \frac{1}{2} \|y - x\|_2^2 + \lambda \|Dx\|_2^2 \\&= \underset{x}{\operatorname{argmin}} \frac{1}{2} \sum_{t=1}^n (y_t - x_t)^2 + \lambda \sum_{t=2}^{n-2} (x_{t-1} - 2x_t + x_{t+1})^2\end{aligned}$$

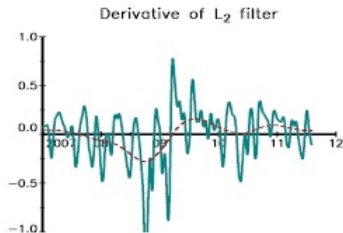
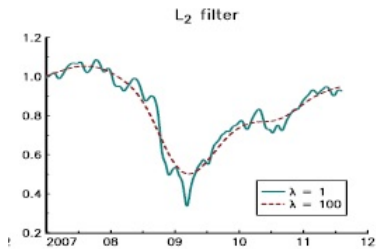
- La solution exacte s'avère être un filtre linéaire :

$$x^* = (I + 2\lambda D^T D)^{-1} y$$

- Estimation de l'hyperparamètre de régularisation λ :
 - par méthode « Bootstrap » ou « Cross-Validation »
 - il existe une méthode plus efficiente en comparant la densité spectrale de ce filtre avec celle d'une moyenne mobile uniforme

Filtres L_2 (2)

Résultats du filtre : tendance x_t et sa dérivée μ_t



Filtre $L_1(1)$

- Résultat du problème d'optimisation suivant :

$$\begin{aligned}x &= \underset{x}{\operatorname{argmin}} \frac{1}{2} \|y - x\|_2^2 + \lambda \|Dx\|_1 \\&= \underset{x}{\operatorname{argmin}} \frac{1}{2} \sum_{t=1}^n (y_t - x_t)^2 + \lambda \sum_{t=2}^{n-2} |x_{t-1} - 2x_t + x_{t+1}|\end{aligned}$$

- **On passe par le problème dual** pour résoudre ce problème
- **Réécriture du problème primal** en introduisant une nouvelle variable $z = Dx$:

$$\begin{aligned}\min_x \quad & \frac{1}{2} \|y - x\|_2^2 + \lambda \|z\|_1 \\s.c. \quad & z = Dx\end{aligned}$$

- En introduisant la variable duale $\nu \in \mathbb{R}^{n-2}$, **le lagrangien s'écrit** :

$$\mathcal{L}(x, z, \nu) = \frac{1}{2} \|y - x\|_2^2 + \lambda \|z\|_1 + \nu^T (Dx - z)$$

Filtre $L_1(2)$

- On peut montrer que le **problème dual** peut s'écrire comme suit :

$$\begin{array}{ll} \max_x & -\frac{1}{2}\nu^T DD^T \nu + y^T D^T \nu \\ \text{s.c.} & -\lambda 1 \leq \nu \leq \lambda 1 \end{array}$$

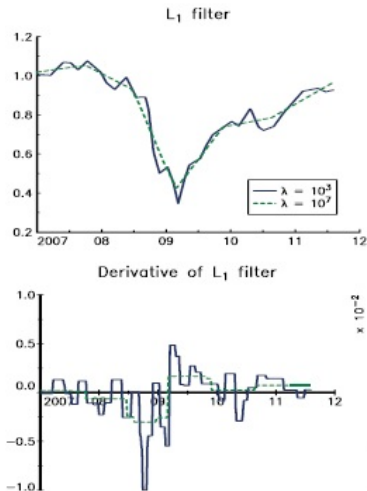
- Il s'agit d'un **problème d'optimisation quadratique** classique résolvable algorithmiquement
- Solution finale du problème primal** : en notant ν^* la solution optimale du problème dual

$$x^* = y - D^T \nu^*$$

- Propriétés :**
 - le filtre L_1 pousse la différentielle seconde de x_t à s'annuler $\Rightarrow$ **signal filtré quasi linéaire par morceaux**
 - plus discriminant que le filtre L_2 sur les changements de tendance
 - l'hyperparamètre λ pilote le nom $\sqrt{\text{bre}}$ de changement de tendance

Filtre $L_1(3)$

Résultats du filtre : tendance x_t et sa dérivée μ_t



Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile
- 5 Filtres L_p
- 6 Filtres de Kalman**
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Représentation de μ_t par filtre de Kalman

- On peut aussi définir la pente μ_t (non observable) de façon indirecte :

$$y_t = x_t + \varepsilon_t$$

$$x_t = x_{t-1} + \mu_t + \eta_t$$

$$\mu_t = \mu_{t-1} + \zeta_t$$

- μ_t est supposée ici **localement linéaire** et les erreurs ε_t , η_t , ζ_t des bruits blancs indépendants
- Ce modèle est un cas particulier des **filtres de Kalman linéaires** :
 - Equation de mesure** :

$$y_t = H_t x_t + d_t + v_t$$

- Equation modélisant les variables cachées x_t** :

$$x_t = A_t x_{t-1} + c_t + w_t$$

- v_t et w_t sont des **bruits blancs indépendants** et de **matrice de covariance respectives R_t et Q_t**

Filtres de Kalman : Notations

- **Variables cachées** x_t : estimation optimale sachant t
 - **Estimateur** : $x_t^a = x_{t|t} = E_t [x_t]$
 - **Erreur d'estimation** : $x_t - x_t^a$
 - **Covariance de l'erreur** : $P_t^a = P_{t|t} = E_t \left[(x_t - x_t^a) (x_t - x_t^a)^T \right]$
- **Variables cachées** x_t : estimation optimale sachant $t - 1$
 - **Estimateur** : $x_t^f = x_{t|t-1} = E_{t-1} [x_t]$
 - **Erreur d'estimation** : $x_t - x_t^f$
 - **Covariance de l'erreur** : $P_t^f = P_{t|t-1} = E_{t-1} \left[(x_t - x_t^f) (x_t - x_t^f)^T \right]$
- **Variables observables** y_t : estimation optimale sachant $t - 1$
 - **Estimateur** : $y_t^f = y_{t|t-1} = E_{t-1} [y_t] = H_t x_t^f + d_t$
 - **Erreur d'estimation** : $z_t = y_t - y_t^f$
 - **Covariance de l'erreur** : $P_t^z = E_t [z_t z_t^T] = E_t \left[(y_t - y_t^f) (y_t - y_t^f)^T \right]$

Filtres de Kalman : Prediction-Correction

- Le filtre de Kalman peut s'estimer par une suite de 2 étapes consécutives :

- Etape de prédiction :**

$$\begin{aligned}x_t^f &= A_t x_{t-1}^a + c_t \\P_t^f &= A_t P_{t-1}^a A_t^T + Q_t\end{aligned}$$

- Etape de correction :** opère une correction linéaire sur la prédiction x_t^f pour tenir compte des observations y_t

$$\begin{aligned}x_t^a &= x_t^f + K_t (z_t - H_t x_t^f - d_t) \\P_t^a &= (I - K_t H_t) P_t^f\end{aligned}$$

- Gain de Kalman :**

$$\begin{aligned}K_t &= P_t^f H_t^T (P_t^z)^{-1} = P_t^f H_t^T (H_t P_t^f H_t^T + R_t)^{-1} \\P_t^z &= H_t P_t^f H_t^T + R_t\end{aligned}$$

- Propriétés :**

- optimal dans le cas d'erreurs gaussiennes
- asymptotiquement stable et insensible aux valeurs initiales

Filtres de Kalman : Estimation des paramètres

- En supposant l'erreur d'estimation $z_t = y_t - y_t^f$ gaussienne et sans biais => **estimation MLE sur un historique de taille n** :

$$\operatorname{argmax} \quad - \sum_{i=1}^n \ln(|P_i^z|) - z_i^T (P_i^z)^{-1} z_i$$

- Les matrices de covariance R_t et Q_t offrent des degrés de liberté redondants => influence similaire sur P_t^z
- **La matrice R_t peut être vue comme une matrice d'hyperparamètres**
- **3 approches en pratique :**
 - R_t fixé à une **matrice diagonale** de variances constantes => hypothèse implicite : invariance du Z-Score de l'erreur $z_t = y_t - y_t^f$
 - **optimisation de l'erreur out-sample** sur un autre historique que celui de l'estimation => effet retard
 - **estimation par un algorithme EM** (estimation bayésienne) => très long

Sommaire

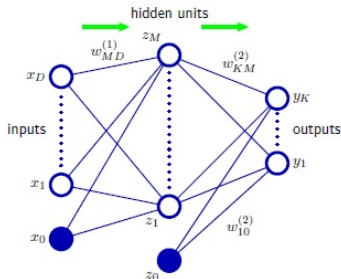
- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile
- 5 Filtres L_p
- 6 Filtres de Kalman
- 7 Méthodes de classification**
- 8 Backtesting et Machine Learning

Classification de la tendance

- **Analyse technique** : méthode empirique de classification supervisée
- **Stratégies tendancielle** : classification d'un historique $X_t = (x_{t-m}, \dots, x_t)$ parmi les 2 classes suivantes
 - C_{up} : tendance haussière
 - C_{down} : tendance baissière
- **Ensemble d'apprentissage** : $D_{\tau} = (X_i, Y_i)_{i \in [1, n]}$ avec $Y_i \in \{0, 1\}$ ou $Y_i \in \{-1, 1\}$ représentant l'une des deux classes C_{up} ou C_{down}
- $Y_i \in \{0, 1\} \Rightarrow$ **outputs homogènes à des probabilités** $\Rightarrow$ signaux de trading directionnels en se fixant des seuils d'activation
- **2 approches possibles pour la détermination de Y_i** :
 - utilisation d'une méthode vue précédemment (régression locale...)
 - digitalisation la série $\Rightarrow$ séquence de taille p consécutive à $X_i \Rightarrow$ application d'une règle simple basée par exemple sur le nombre de hausses/baisses consécutives
- **Principales approches (non-linéaires)** :
 - réseaux de neurones
 - Support Vector Machines (SVM)

Réseaux de neurones : Topologie

- Approche née dans les années 60 avec la tentative de modéliser mathématiquement le fonctionnement du cerveau
- Réalisme biologique délesté depuis => **série de convolutions imbriquées de fonctionnelles non-linéaires**
- **Réseau de neurones à une couche cachée** (topologie « Feed-Forward ») :
 - **Input** : $X = (x_1, \dots, x_D)$
 - **Couche cachée** : $Z = (z_1, \dots, z_M)$
 - **Output** : $Y = (y_1, \dots, y_K)$



Réseaux de neurones : Principes

- **Principes :**

- les inputs d'un neurone sont tous les outputs de la couche précédente
- la valeur de sortie d'un neurone est la composée d'une fonction d'activation avec la combinaison linéaire de tous ses inputs

- **Output du neurone $j \in [1, M]$ de la couche cachée :**

$$z_j = \phi_j \left(\sum_{l=1}^D w_{j,l}^{(1)} x_l + w_{j,0}^{(1)} \right)$$

- **Output du neurone $k \in [1, K]$ de la couche de sortie :**

$$y_k = f \left(\sum_{j=1}^M w_{k,j}^{(2)} \phi_j \left(\sum_{l=1}^D w_{j,l}^{(1)} x_l + w_{j,0}^{(1)} \right) + w_{k,0}^{(2)} \right)$$

- En introduisant un input fictif $x_0 = 1$ et un neurone caché $z_0 = 1$:

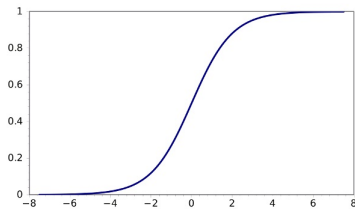
$$y_k = f \left(\sum_{j=0}^M w_{k,j}^{(2)} \phi_j \left(\sum_{l=0}^D w_{j,l}^{(1)} x_l \right) \right)$$

Réseaux de neurones : Variantes et propriétés

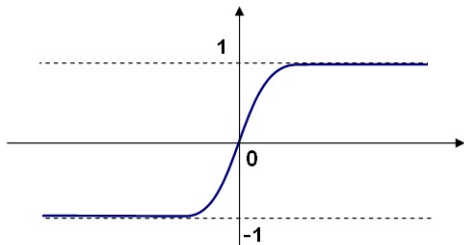
- **Variantes :**

- Plusieurs couches cachées
- **Sparse Layer** : on ne connecte pas tous les neurones entre eux
- **Skip Layer** : pour certains neurones, on peut sauter les connections à la couche précédente
- La complexité provient de la composition successive des fonctionnelles d'activation
- **Approximateur universel** : on peut approximer n'importe quelle fonction vectorielle à la précision voulue du moment que la couche cachée comporte suffisamment de neurones
- Les **fonctions d'activation** $(\phi_j)_{j \in [1, M]}$ et f sont généralement :
 - des **fonctions sigmoïdes** si $Y_i \in \{0, 1\}$
 - des **tangentes hyperboliques** si $Y_i \in \{-1, 1\}$

Réseaux de neurones : Fonctions d'activation



(a) Sigmoïde



(b) Tangente Hyperbolique

Figure – Fonctions d'activation

Réseaux de neurones : Estimation (1)

- **Paramètres à estimer :**

- poids reliant chaque neurone
- paramètres éventuels des fonctions d'activation

- **Critère d'estimation :** minimisation d'une fonction de perte $E[w]$

- **Définitions usuelles pour $E[w]$:**

- MCO :

$$E[w] = \sum_{i=1}^n (f(X_i) - Y_i)^2$$

- (-) **MLE :** la loi $p(Y | X) = (f(X))^Y (1 - f(X))^{1-Y}$ suit une loi de bernouilli

$$E[w] = - \sum_{i=1}^n Y_i \ln(f(X_i)) + (1 - Y_i) \ln(1 - f(X_i))$$

- **Regularisation possible de la fonction de perte :**

$$\tilde{E}[w] = E[w] + \frac{\lambda}{2} \|w\|^2$$

- **Algorithmes utilisés :** descente de gradient, gradient-conjugué, quasi-Newton...

Réseaux de neurones : Estimation (2)

- **Problèmes d'estimation :**

- **Dimension du problème élevé** $\Rightarrow$ existence de plusieurs locaux $\Rightarrow$ plusieurs minimisations à partir de différents points de départ afin de trouver le minimum global
- **Redondances** liées à la symétrie des fonctions d'activation sigmoïdes et tangentes hyperboliques ($f(-x) = 1 - f(x)$)

- **Amélioration de la convergence et des temps de calculs :**

- au lieu de modifier le vecteur de poids $w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E [w^{(\tau)}]$ à chaque étape de l'algorithme d'optimisation, il est plus efficace de le modifier séquentiellement sur chaque élément de D_τ :

$$w^{(\tau+1)} = w^{(\tau)} - \eta \nabla E_i [w^{(\tau)}]$$

- « **Error Backpropagation** » à la place des différences finies pour le calcul des gradients et de la hessienne
- **Error Backpropagation** = utilisation massive de dérivations en chaîne (Chain Rule)

Réseaux de neurones : Error Backpropagation (1)

- Illustration de la méthode à travers le **calcul du gradient**
- En introduisant les **variables d'activation** $\left(a_j^{(i)}\right)_{j \in [0, M]}$ et $a^{(i)} = a_K^{(i)} = a_1^{(i)}$:

$$\begin{aligned} Y_i &= f(X_i) = f\left(\left(x_1^{(i)}, \dots, x_D^{(i)}\right)\right) \\ &= f\left(\sum_{j=0}^M w_j^{(2)} \phi_j\left(a_j^{(i)}\right)\right) \\ &= f\left(a^{(i)}\right) \end{aligned}$$

$$\begin{aligned} a_j^{(i)} &= \sum_{l=0}^D w_{j,l}^{(1)} x_l^{(i)} \\ a^{(i)} &= \sum_{j=0}^M w_j^{(2)} \phi_j\left(a_j^{(i)}\right) \end{aligned}$$

Réseaux de neurones : Error Backpropagation (2)

- Dérivées partielles par rapport à $w_j^{(2)}$:

$$\begin{aligned}\frac{\partial E_i[w]}{\partial w_j^{(2)}} &= \frac{\partial E_i[w]}{\partial a^{(i)}} \frac{\partial a^{(i)}}{\partial w_j^{(2)}} \\ &= \delta_i \frac{\partial a^{(i)}}{\partial w_j^{(2)}} \\ &= \delta_i \phi_j \left(a_j^{(i)} \right)\end{aligned}$$

- Pour le critère MCO, δ_i est linéaire en l'erreur du réseau :

$$\delta_i = 2 \left(f(X_i) - Y_i \right)$$

Réseaux de neurones : Error Backpropagation (3)

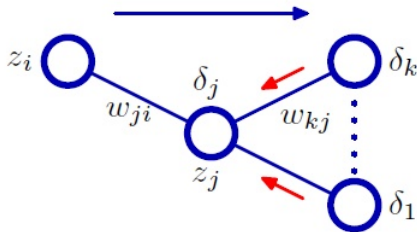
- Dérivées partielles par rapport à $w_{j,l}^{(1)}$:

$$\begin{aligned}\frac{\partial E_i[w]}{\partial w_{j,l}^{(1)}} &= \frac{\partial E_i[w]}{\partial a_j^{(i)}} \frac{\partial a_j^{(i)}}{\partial w_{j,l}^{(1)}} \\ &= \delta_{i,j} \frac{\partial a_j^{(i)}}{\partial w_{j,l}^{(1)}} \\ &= \delta_{i,j} x_l^{(i)}\end{aligned}$$

$$\begin{aligned}\delta_{i,j} &= \frac{\partial E_i[w]}{\partial a_j^{(i)}} \\ &= \frac{\partial E_i[w]}{\partial a^{(i)}} \frac{\partial a^{(i)}}{\partial a_j^{(i)}} \\ &= \delta_i w_j^{(2)} \phi_j' \left(a_j^{(i)} \right)\end{aligned}$$

Réseaux de neurones : Error Backpropagation (4)

Le calcul de $\delta_{i,j}$ s'effectue ainsi en rétropropageant l'erreur du réseau via $\delta_i = 2(f(X_i) - Y_i)$



Réseaux de neurones : Analyse

- **Avantages :**

- **Approximateur universel** : méthode de classification non-linéaire la plus générale
- **Apprentissage très souple** : le réseau apprend de lui même à partir de l'ensemble d'apprentissage
- **Possibilité d'ajouter des variables exogènes** décrivant le contexte de marché afin d'améliorer l'apprentissage et la qualité des signaux générés (Machine Learning)

- **Inconvénients :**

- **Nombre élevé des paramètres** => problème d'estimation + risque d'over-fitting
- **Nombreux minimums locaux**
- **Instabilités** : les solutions trouvées dépendent très souvent de l'ordre d'apprentissage

Support Vector Machine : Presentation

- **Principales caractéristiques :**

- Méthode de **classification non-linéaire à deux classes**
- **Moins générale que les réseaux de neurones**
- **Aboutit à la résolution d'un problème quadratique** convexe lors de l'estimation des paramètres

- La frontière de décision est initialement non-linéaire mais **on se ramène au cas linéaire par une transformation de l'espace :**

$$Y = f(X) = w^T \phi(X) + b$$

- $Y \in \mathbb{R}$ mais les valeurs de notre ensemble d'apprentissage $Y_i \in \{-1, 1\}$
- La frontière de décision recherchée étant définie par $f(X) = 0 \Rightarrow$ on doit avoir pour tout élément de D_τ :

$$\begin{cases} f(X_i) > 0 & \text{si } Y_i = 1 \\ f(X_i) < 0 & \text{si } Y_i = -1 \end{cases}$$

- Autrement dit : $Y_i f(X_i) > 0$

Support Vector Machine : Frontière de décision

- **SVM définit la frontière de décision de façon indirecte** : c'est l'hyperplan tel que la distance à n'importe quel point X_i de D_τ et en particulier l'élément le plus proche, soit maximale
- On peut montrer que cette distance s'écrit comme suit :

$$\frac{|f(X_i)|}{\|w\|} = \frac{Y_i f(X_i)}{\|w\|} = \frac{Y_i (w^T \phi(X_i) + b)}{\|w\|}$$

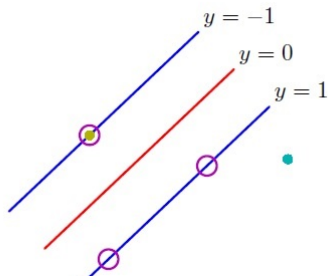
- **Frontière de décision** :

$$\underset{w,b}{\operatorname{argmax}} \left\{ \min_i \frac{Y_i (w^T \phi(X_i) + b)}{\|w\|} \right\}$$

Support Vector Machine : Hyperplans et vecteurs support

- **Problème invariant pour toute homothétie conjointe sur (w, b)**
- **Hyperplans supports** : on peut lever ce degré de liberté en fixant arbitrairement l'équation des 2 hyperplans parallèles à la frontière de décision $f(X) = 0$
- **Vecteurs de support** : ces 2 hyperplans passent par les points X_{i^*} les plus proches de $f(X) = 0$

$$Y_{i^*} (w^T \phi(X_{i^*}) + b) = 1$$



Support Vector Machine : Problème quadratique

- On est donc amené à résoudre le **problème quadratique sous contraintes** suivant :

$$\begin{array}{ll} \underset{w, b}{\operatorname{argmax}} & \frac{1}{\|w\|} \\ \text{s.c.} & Y_i (w^T \phi(X_i) + b) \geq 1 \end{array}$$

- Ou de façon équivalente :

$$\begin{array}{ll} \underset{w, b}{\operatorname{argmin}} & \frac{1}{2} \|w\|^2 \\ \text{s.c.} & Y_i (w^T \phi(X_i) + b) \geq 1 \end{array}$$

- Lagrangien** de ce problème primal :

$$\mathcal{L}_{\mathcal{P}}(w, b, a) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n a_i (Y_i (w^T \phi(X_i) + b) - 1)$$

Support Vector Machine : Problème dual et kernels

- **Problème dual :**

$$\begin{aligned} \underset{a}{\operatorname{argmax}} \quad & \mathcal{L}_{\mathcal{D}}(a) = \sum_{i=1}^n a_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_i a_j Y_i Y_j k(X_i, X_j) \\ \text{s.c.} \quad & a_i \geq 0 \\ \text{s.c} \quad & \sum_{i=1}^n a_i Y_i = 0 \end{aligned}$$

- La fonction « noyau » (kernel) définie par $k(X_1, X_2) = \phi(X_1)^T \phi(X_2)$ est déterminante dans la résolution de ce problème dual
- On préfère souvent directement définir le modèle par ce kernel
- **Kernels les plus utilisés :**
 - **Kernel polynomial :** $k(X_1, X_2) = (X_1^T X_2 + c)^p$
 - **Kernel gaussien :** $k(X_1, X_2) = \exp\left(-\frac{\|X_1 - X_2\|_2^2}{2\sigma^2}\right)$
 - **Kernel sigmoïdal :** $k(X_1, X_2) = \tanh(aX_1^T X_2 + c)$

Support Vector Machine : Solution (1)

- Ce problème dual est **également un problème quadratique**
- **Défaut apparent** : ce problème dual implique cette fois-ci $n > m$ variables
- **Avantages réels** :
 - le kernel régularise énormément la fonction à optimiser
 - conditions de Kuhn-Tucker $\Rightarrow a_i \bar{s} = 0$.
- **La frontière de décision ne dépend que des vecteurs de support** :

$$f(X) = \sum_{i \in S} a_i Y_i k(X, X_i) + b$$

- **Critère de décision** :

$$\begin{cases} f(X_i) > 0 & C_{up} \\ f(X_i) < 0 & C_{down} \end{cases}$$

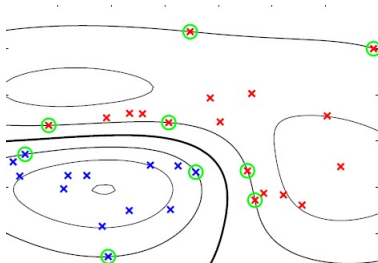
Support Vector Machine : Solution (2)

- Pour tout vecteur de support $X_{j \in S}$, on a :

$$Y_j \left(\sum_{i \in S} a_i Y_i k(X_j, X_i) + b \right) = 1$$

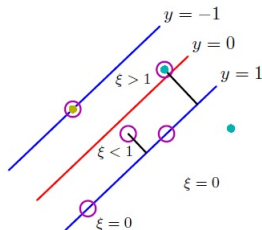
- **Calcul de la constante b** : pour plus de stabilité, on moyenne sur tous les vecteurs de support :

$$b = \frac{1}{n_S} \sum_{j \in S} \left(Y_j - \sum_{i \in S} a_i Y_i k(X_j, X_i) \right)$$



Support Vector Machine : Extension (1)

- **On tolère des erreurs de classification dans $D_\tau \Rightarrow$ l'ensemble d'apprentissage n'est plus linéairement séparable dans l'espace paramétré par $\phi(X)$**
- **Variables auxiliaires $\xi_i \geq 0$: erreur de classification opérée par le modèle**
- **3 cas :**
 - $\xi_i = 0$: pas d'erreur, le point est correctement classifié
 - $0 < \xi_i \leq 1$: le point est correctement classifié mais est situé entre la frontière de décision ($\xi = 0$) et l'un des 2 hyperplans supports
 - $\xi_i > 1$: le point n'est pas correctement classifié



Support Vector Machine : Extension (2)

- La contrainte $Y_i f(X_i) \geq 1$ s'écrit ici :

$$Y_i (w^T \phi(X_i) + b) \geq 1 - \xi_i$$

- Problème quadratique sous contraintes :**

$$\begin{aligned} \underset{w, b}{\operatorname{argmax}} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.c.} \quad & Y_i (w^T \phi(X_i) + b) \geq 1 - \xi_i \end{aligned}$$

- Même problème dual** mais les a_i sont cette fois-ci bornés par C :

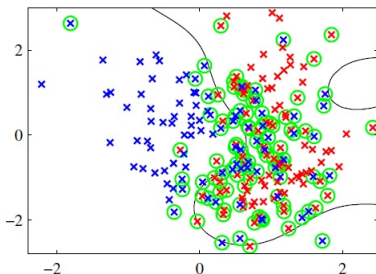
$$\begin{aligned} \underset{a}{\operatorname{argmax}} \quad & \mathcal{L}_{\mathcal{D}}(a) = \sum_{i=1}^n a_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a_i a_j Y_i Y_j k(X_i, X_j) \\ \text{s.c.} \quad & 0 \leq a_i \leq C \\ \text{s.c} \quad & \sum_{i=1}^n a_i Y_i = 0 \end{aligned}$$

Support Vector Machine : Extension (3)

- **Même solution que dans le cas séparable :**

$$f(X) = \sum_{i \in S} a_i Y_i k(X, X_i) + b$$

- **Néanmoins ici 2 sous-ensembles de vecteurs de support :**
 - $0 < a_i < C$: points définissant les deux hyperplans supports ($\xi_i = 0$)
 - $a_i = C$: points tels que $\xi_i > 0$



Support Vector Machine : Analyse

- **Défauts :**

- **Moins générale que les réseaux de neurones** (n'est pas un approximateur universel)
- **Succès de l'approche dépendant directement du pourcentage de vecteurs supports retenus** : si ce pourcentage est trop élevé, le risque d'over-fitting devient important

- **Avantages :**

- **Problème d'estimation quadratique** $\Rightarrow$ convergence vers le minimum global
- **Stabilité et robustesse** : la classification ne dépend que d'un nombre limité de vecteurs de support

Sommaire

- 1 Analyse graphique et reconnaissance de forme
- 2 Représentation temporelle : régression locale et méthodes de lissage
- 3 Représentation fréquentielle : méthodes de seuillage
- 4 Filtres à moyenne mobile
- 5 Filtres L_p
- 6 Filtres de Kalman
- 7 Méthodes de classification
- 8 Backtesting et Machine Learning

Classification : Feature Engineering (1)

- **Input vector**

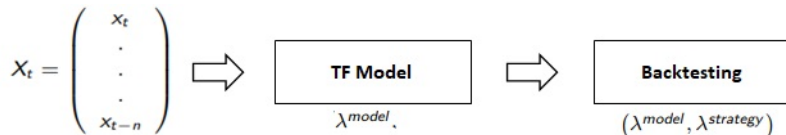
- **Signal Digitalisation or Transformation** => reduce as much as possible the complexity (Binary Strings...)
- **Features vector**

$$X_t = (Momentum_t, Volatility_t, MaxDrawDown_t, FinancialIndicators_t, ...)$$

- **TF models can be viewed as a Feature Engineering procedure within a classification model** => excepting dynamic models (ex : Kalman filtering...)

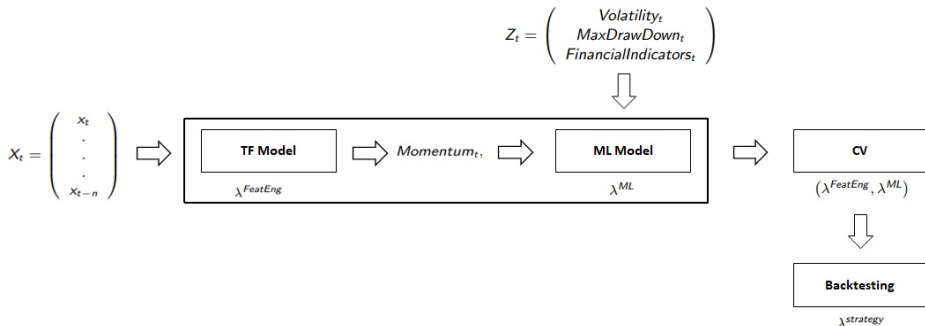
Classification : Feature engineering (2)

- « **Rules-Based** » **approach** hyperparameters and strategy parameters $(\lambda^{model}, \lambda^{strategy})$ are all calibrated through Backtesting



Classification : Feature engineering (3)

- « **Classification** » approach hyperparameters $(\lambda^{FeatEng}, \lambda^{ML})$ are calibrated through CV and strategy parameters $\lambda^{strategy}$ through Backtesting



Classification : Feature engineering (4)

- In the same way, **Statistical Arbitrage models can also be reconfigured in a Classification Framework** => explanatory variables are added to the input vector
- **Cross-Sectional approach** => applied on the whole trading universe
 - **ZScoring or Ranking scheme** => applied both on input features and output metric (return, performance measure...) => multi-class label
 - **Better results than the « Absolute Value » approach**

Classification : Labeling (1)

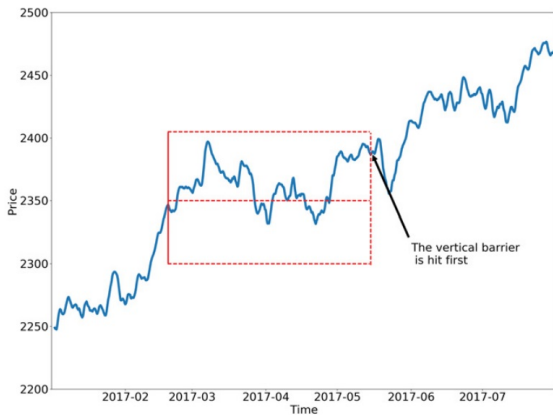
- **Fixed-Time Horizon Labeling**

$$y_i = \begin{cases} -1 & \text{if } r_{t_{i,0}, t_{i,0}+h} < -\tau \\ 0 & \text{if } |r_{t_{i,0}, t_{i,0}+h}| \leq \tau \\ 1 & \text{if } r_{t_{i,0}, t_{i,0}+h} > \tau \end{cases} \quad r_{t_{i,0}, t_{i,0}+h} = \frac{p_{t_{i,0}+h}}{p_{t_{i,0}}} - 1$$

- **Trend-Scanning labeling** => ex-post use of a TF model (Binary String, Linear Filters, temporal or frequencial model...)

Classification : Labeling (2)

- **Triple-Barrier Labeling** 3 barriers are set in terms of a number of strategy's standard deviations
 - 1 **Upper Horizontal** (Stop-Gain)
 - 2 **Lower Horizontal** (Stop-Loss)
 - 3 **Vertical Barrier** (Stop-Horizon)

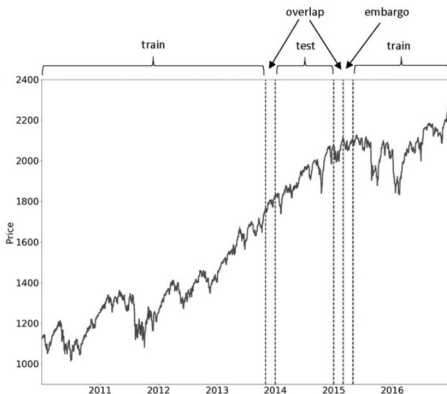


CV & Backtesting

- **CV practical use** => model selection through the calibration of hyperparameters (which may include various models)
- **Backtesting is the « old-fashioned » financial way of CV** => they broadly differ in their criteria
 - **Cross-Validation** => Minimize a Loss Function metric (static)
 - **Backtesting** => Maximizing a Performance metric (dynamic)
- **CV & Backtesting practical use**
 - « **Rules-Based** » approach hyperparameters and strategy parameters ($\lambda^{model}, \lambda^{strategy}$) are all calibrated through Backtesting
 - « **Classification** » approach hyperparameters ($\lambda^{FeatEng}, \lambda^{ML}$) are calibrated through CV and strategy parameters $\lambda^{strategy}$ through Backtesting

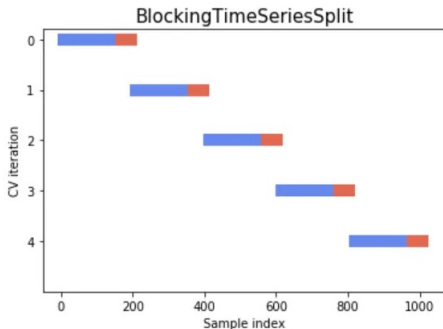
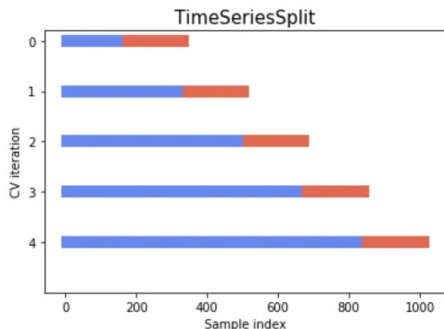
CV in Finance

- **Classical K-Fold CV cannot be directly used**
 - **Observations are not IID** $\Rightarrow$ due to high autocorrelation
 - **Leakage appears when the training set contains information also present in the Test set** $\Rightarrow$ overlapping data
- **Purge K-Fold CV**



Backtesting approaches (1)

- **Walk-Forward Method** => only 1 path !
- **Times Series Method** => slightly better out-Sample estimator & no overlapping/autocorrelation issues



Backtesting approaches (2)

- **Purge K-Fold Method** => still one path but better out-sample estimator
- **Combinatorial Purge K-Fold Method** => many paths
 - ① split your historical in N periods with a size enough to perform a backtest on each
 - ② define the number k of periods for the Test set => n-k periods for the Training set

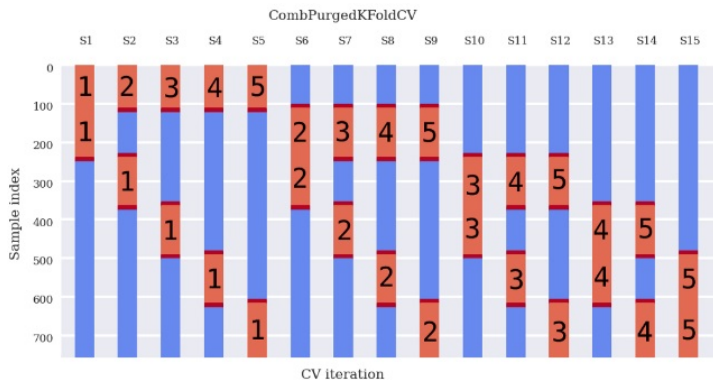
$$\binom{N}{N-k} = \frac{\prod_{i=0}^{k-1} (N-i)}{k!}$$

- **Number of combinatorial splits**
- **Number of Backtests paths generated**

$$\varphi[N, k] = \frac{k}{N} \binom{N}{N-k} = \frac{\prod_{i=1}^{k-1} (N-i)}{(k-1)!}$$

Backtesting approaches (3)

- $N = 6$ and $k = 2 \Rightarrow 5$ backtest paths



Event Sampling

- **Idea** => filter output signals to improve accuracy and performance
- **Type of filters**
 - **Event-based** => filter output signals to improve accuracy and performance
 - **Structural breaks** => Mann-Kendal test, CUSUM filter
- **CUSUM filter** => identify upside or downside divergence

$$\begin{cases} S_t^+ = \max(0, S_{t-1}^+ + y_t - E_{t-1}[y_t]) & S_0^+ = 0 \\ S_t^- = \max(0, S_{t-1}^- + y_t - E_{t-1}[y_t]) & S_0^- = 0 \end{cases}$$

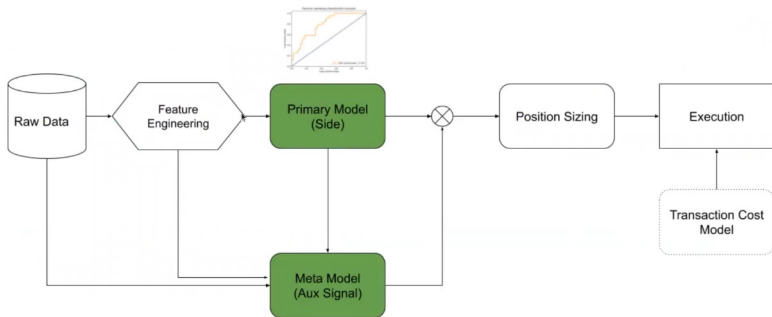
$$\begin{cases} S_t = \max(S_t^+, S_t^-) \\ S_t \geq h \end{cases}$$

Meta-Labeling (1)

- **Idea** => train a second model using first model outputs, to determine if we should trade the signal or not => improve model recall (true positives)
- **Meta-Labels** => labels first model outputs as correct or incorrect (ex-post)
- **Typical Features added to the framework** => include the Market context
 - ① Primary model features
 - ② Market state
 - ③ Recent model performances

Meta-Labeling (2)

- More complex architectures



Sizing (1)

- **Sizing Schemes**

- **Fixed Capital amount** => ex : 100 dollars on each trade
- **Fixed Risk Budget** => Naive Risk Parity $\frac{1}{\sigma_i}$
- **Abs(ZScore) weighting** => signal generation
- **Probability weighting** => explicit or implicit Classifier Output (a binary label could always be translated into a probability using an Isotonic or Sigmoid transformation)
- **Kelly Criterion** => well-known by gamblers (ex : Poker...). For a given probability of success **p** (estimated through backtesting) :

$$OptimalBetSize = \frac{p}{\%Loss} - \frac{1-p}{\%Gain}$$

Sizing (2)

- **Leverage**

- **Autofinanced strategy** => weights must sum to 1 and rebalancing (with related transaction costs) must be carefully managed
- **Non Autofinanced strategy** => weights don't necessarily sum to 1 => borrowing & lending costs must be taken into account !

- **Instrument Type**

- **Asset** => positions corresponds to real money
- **Derivatives** => positions correspond to an exposure (the entry price is ended zero and initial margin implicitly defines the implied Leverage)

- **Treasury must be carefully managed when Backtesting !**